Source-Driven E2E Test Generation over a Fixed Workflow Space: An Angular Case Study

Anonymous Author(s)

Anonymous Institution

Abstract. End-to-end (E2E) functional testing of web applications is commonly assessed through coverage, but existing approaches compute coverage over interaction spaces derived from external models, runtime exploration, or textual specifications. The resulting denominators reflect abstraction fidelity or exploration budget rather than the executable behavior encoded in the frontend implementation. This paper presents a source-driven, multi-phase pipeline that constructs a fixed, constraint-aware coverage denominator directly from Angular source code. Phase A statically extracts a UI Interaction Multigraph encoding navigation structure, widget interactions, and constraint surfaces, then enumerates one augmented workflow per user-interaction trigger to produce a finite workflow space. Phase B realizes every workflow into an executable Selenium test via manifest-driven constraint binding, generates deterministic test code with embedded observability instrumentation, and executes each test with integrity-verified evidence collection. Evaluation on six open-source Angular applications yields a workflow space of 257 augmented workflows with 100% plan and code generation coverage. Execution coverage on five integrity-verified subjects reaches 64.7% (156/241 tests passing), with a systematic failure taxonomy classifying 85 residual failures into seven families—74% attributable to asynchronous widget materialization rather than to workflow construction or test generation deficiencies.

Keywords: End-to-End Testing · Workflow Coverage · Source-Driven Testing · Angular · Static Analysis · Selenium · Test Generation · Coverage Denominator · Observability

1 Introduction

End-to-end (E2E) functional testing validates web applications by executing sequences of user interactions against the running system [1]. For modern applications built with component-based frameworks (e.g., Angular[1], React[2]) user-visible behavior is defined primarily at the frontend through routing configurations, component templates, event handlers, and guard-protected navigation paths [8,13,2], making E2E testing essential for detecting defects at this level.

[1] https://angular.dev/
[2] https://react.dev/

The effectiveness of E2E testing is commonly assessed through coverage [8,28]: the proportion of an application's interaction behavior exercised by the test suite with respect to a reference space that defines which behaviors count [27]. In this setting, a workflow is a sequence of user interactions that exercises a frontend path. However, structural reachability does not imply executability: a workflow may remain infeasible if required guards are unsatisfied, route parameters are missing, form constraints fail, or visibility predicates evaluate to false. We refer to the collection of such conditions as the workflow's *constraint surface*. Ignoring constraint surfaces inflates the reference space with infeasible workflows and therefore distorts coverage claims [28,19].

Existing approaches construct interaction spaces either statically or dynamically. Static approaches, including model-based testing [27,7], artifact-driven derivation [20,29], and LLM-based synthesis [11,3,10,12], derive coverage from external models or specifications. Dynamic approaches, including crawling [17,18], exploration [31,1], and LLM-driven agents [4,24], discover behavior at runtime but are bounded by exploration budget and non-deterministic. Across both families, coverage is evaluated over derived representations rather than over a fixed, implementation-grounded workflow space.

This leaves a gap: to the best of our knowledge, no existing approach constructs a finite, constraint-aware reference space of frontend interaction behavior directly from source code and uses that space as the coverage denominator.

In this paper, we address this gap. Our research objective is to maximize the coverage of automatically generated, executable E2E functional tests for Angular web applications through a source-driven, constraint-aware, multi-phase pipeline, and to characterize the structural and runtime limits of that coverage through empirical evaluation. We pursue three research questions:

RQ1. How can executable frontend interaction behavior be represented and deterministically constructed from Angular source code as a finite, implementation-grounded workflow space?

RQ2. How can workflows in that space be realized into executable Selenium tests with full generation coverage (i.e., a generated test for every workflow)?

RQ3. How effective is the approach empirically, relative to a dynamic crawling baseline, and what residual failure families delimit its current boundary?

To answer these questions, we present a two-phase pipeline. Phase A statically extracts a *UI Interaction Multigraph* from Angular source code, enumerates one workflow per user-interaction trigger, classifies each by constraint feasibility, and yields a fixed workflow space serving as the coverage denominator. Phase B binds symbolic constraints to runtime-specific values via a per-subject manifest, derives executable test plans, generates Selenium test code for every workflow, executes each test with structured observability and integrity verification, and computes tiered coverage metrics over the fixed denominator.

This paper contributes:

1. A *UI Interaction Multigraph* formalism (six node kinds, eighteen edge kinds) that represents Angular navigation, interaction, and constraint structure as a single deterministic, source-traceable artifact.

2. A *trigger-centric workflow enumeration* algorithm that constructs a finite, classified workflow space yielding a fixed coverage denominator.
3. A *five-stage realization pipeline* that transforms every workflow into an executable Selenium test with structured observability and integrity-verified coverage computation.
4. An *empirical evaluation* on six Angular applications: 100% generation coverage (257/257 workflows), 64.7% execution coverage on five subjects under integrity-verified conditions (156/241 passing), a seven-family failure taxonomy, and a comparison against AutoE2E [1].

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 presents the approach. Section 4 describes the evaluation. Section 5 concludes.

2 Related Work

We organize related work into four families and compare them by interaction-space source, coverage semantics, and behavioral granularity.

Model-based and specification-driven testing. MBT derives test suites from behavioral models (e.g., finite state machines) using model-defined coverage criteria [27,28,7]. For web applications, models are built from specifications or artifacts such as routing configurations [19]; a related line derives scenarios from use case specifications [20], requirements [6], or UML models [29]. The coverage denominator is the model or specification, whose fidelity to the implementation is not guaranteed: guards, role constraints, and visibility conditions are typically abstracted away. Granularity is at the page, state, or route level; widget interactions, handler effects, and constraint surfaces are not represented.

Source-driven extraction. A complementary line applies static analysis directly to source code. Jensen et al. [9] and Madsen et al. [15] establish that sound static reasoning about JavaScript control/data flow is feasible, including framework-aware abstractions, but target general program properties rather than interaction-space construction. Closer to testing, JSDEP [26] computes DOM event-handler dependencies to guide the ARTEMIS platform, and Park et al. [22] analyze event-handler interactions with visibility state, though relying on dynamically collected states. These operate at event/handler granularity but do not aggregate constraint surfaces or enumerate workflows. For SPA verification, Oshima and Kishi [21] extract page-transition state machines from Angular source for model checking, and Zhang et al. [30] extract navigational graphs with guard annotations from SPA bundles for access-control verification. Both recover route-level structure but neither models widget-level interactions, computes handler-effect closures, or aggregates constraint surfaces—and therefore cannot distinguish feasible from infeasible workflows. Neither produces executable tests or defines a coverage denominator.

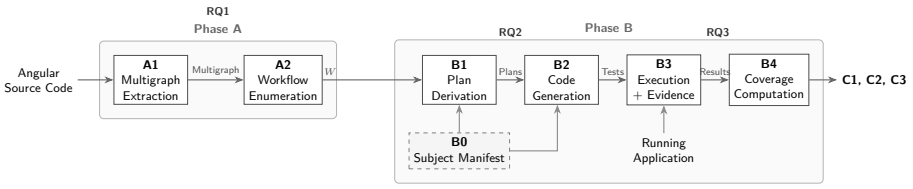

Fig. 1. Phase A extracts a deterministic multigraph and enumerates W. Phase B realizes every workflow into a Selenium test with structured observability and computes tiered coverage.

Crawling and runtime exploration. Dynamic approaches construct models by executing the application. Crawling [17,18], RL-guided exploration [31,25,14], GUI ripping [16], and scriptless testing [5,23] discover states and transitions at the level of DOM-level transitions. The interaction space is exploration-bounded and non-deterministic, yielding variable coverage denominators across runs. Authentication walls further limit exploration to unauthenticated regions [1].

LLM-based test generation and web agents. One line synthesizes test scripts from textual artifacts [11,3,10,12], deriving the interaction space from input text. A second employs LLM-driven agents: AutoE2E [1] combines crawling with feature inference; others execute natural-language instructions [4] or reason iteratively toward targets [24]. Granularity is at the feature or task-description level. Both generation and execution are non-deterministic, and the interaction space is not grounded in source-level structure.

Positioning. Prior source-driven work shows that extraction from frontend code is feasible at multiple granularity levels [9,15,26,21,30]. However, we are not aware of prior work combining: (1) widget-level interaction extraction from component templates and handler bodies; (2) handler-effect closure computation; (3) constraint-surface aggregation; (4) finite workflow enumeration yielding a fixed coverage denominator; and (5) end-to-end realization into executable tests with integrity-verified coverage computation. Our work is positioned at this combination: it extracts a constraint-aware workflow space from Angular source code at widget-and-handler granularity, produces one executable test per workflow, and computes coverage over a fixed denominator.

3 Approach

3.1 Overview

Our approach takes Angular frontend source code as input and constructs a fixed workflow space W that serves as the coverage denominator for E2E test generation and execution. The pipeline consists of two phases (Fig. 1).

Phase A constructs W from source code alone. A1 statically analyzes the Angular AST and constructs a *UI Interaction Multigraph*—an intermediate representation whose nodes represent implemented entities (modules, routes, components, widgets, services, external targets) and whose edges encode structural

relations and executable transitions with attached constraint surfaces. A2 derives W by enumerating one workflow per enabled trigger edge, classifying each as FEASIBLE, CONDITIONAL, or PRUNED. Both stages are deterministic; Phase A addresses **RQ1**.

Phase B operates over the fixed W without modifying it. B0 supplies runtime-specific values via a per-subject manifest. B1 derives one action plan per workflow; B2 generates one Selenium test per plan; B3 executes each test with integrity-verified observability; B4 computes tiered coverage (C1, C2, C3) over W. Phase B addresses **RQ2** (B0–B2) and **RQ3** (B3–B4).

3.2 A1: UI Interaction Multigraph

The multigraph is a deterministic, source-derived intermediate representation encoding navigation structure, interaction behavior, and constraint surfaces. It provides an implementation-grounded structure where every transition carries its constraint surface and every element is traceable to source code, distinguishing reachable paths from infeasible ones. Formally, it is a typed directed graph $G = (V, E)$.

Node kinds. V contains six kinds: **Module** (declaration scope, providers), **Route** (navigation target with path template and guards), **Component** (template owner, handler binder), **Widget** (interaction site—button, form, input—and atomic trigger unit), **Service** (injectable cross-cutting behavior), and **External** (off-application URL). Guards are constraint metadata on routes and edges, not nodes.

Edge kinds. Structural edges (eleven kinds) encode containment, declaration, hierarchy, and provider relationships. *Executable edges* (seven kinds) encode runtime transitions: four originate at widgets (navigate, trigger handler, submit form); three at components or routes (call service, navigate, redirect). Composite behavior is represented as ordered edges sharing an effect group identifier. The four widget-origin kinds are the *trigger edges* enumerated by A2.

Constraints, traceability, determinism. Each executable edge carries a *constraint surface*: guard names, required route parameter keys, and UI precondition atoms (visibility, enablement, form validity). Every node and edge carries source-location references. Extraction is deterministic: identical input yields byte-identical output with stably sorted identifiers. A1 produces `a1-multigraph.json`, frozen as immutable input to all subsequent stages.

3.3 A2: Workflow Enumeration and Classification

A2 transforms G into W: a finite, deterministic set of classified workflows constituting the coverage denominator, completing RQ1. It establishes a fixed denominator defined before any test is generated, provides a complete workflow-level abstraction (trigger + effects + constraints), distinguishes executable from infeasible behavior via classification, and bounds $|W|$ by construction (one workflow per enabled trigger).

Algorithm 1 Augmented Workflow Enumeration and Classification

Require: Frozen multigraph $G = (V, E)$
Ensure: Workflow space W

```
W ← ∅; seen ← ∅                                  ▷ seen tracks trigger edge IDs for aggregation
for all component-bearing route r ∈ G, sorted by id do
    A_c ← ActiveComponents(r)                    ▷ route + ancestors + composition closure
    A_w ← ActiveWidgets(A_c)                     ▷ widgets owned by active components
    for all trigger edge t from widgets in A_w, sorted by edge id do
        if t.from is statically hidden or disabled then skip
        end if
        if t.id ∈ seen then
            append r to W[t.id].startRouteIds    ▷ route aggregation
            continue
        end if
        steps ← [t]
        if t has effectGroupId then
            append CCS edges (sorted by callsite ordinal)
            append CNR edge (if present, at most one)
        end if
        terminal ← ResolveTerminal(t, steps)     ▷ redirect closure, cycle detection
        C(w) ← MergeConstraints(steps)           ▷ union of guards, params, atoms, evidence
        verdict ← Classify(C(w))                 ▷ PRUNED → CONDITIONAL → FEASIBLE
        W ← W ∪ {(t.id, steps, terminal, C(w), verdict)}
        seen ← seen ∪ {t.id}
    end for
end for
return W
```

Augmented workflow. An *augmented workflow* w consists of a trigger edge t, an ordered effect closure $[s_1, \ldots, s_k]$ of handler-scoped service calls and navigation, and an aggregated constraint surface $C(w)$. Each workflow records start route(s), terminal node, and evidence metadata.

Trigger-centric enumeration. For each component-bearing route r (sorted by id), A2 computes *active components* (activated by r and ancestors) and *active widgets*. A trigger edge is *enabled* if its widget is active and not unconditionally hidden or disabled. Exactly one workflow is constructed per enabled trigger; shared-component triggers are aggregated. Algorithm 1 summarizes the procedure.

Effect closure and constraint aggregation. When a trigger carries an effect group identifier, A2 collects matching edges: service calls (sorted by callsite ordinal) and at most one navigation edge. Navigation targets undergo *redirect closure* with cycle detection. $C(w)$ merges per-edge surfaces: parameter keys and guards by set union, UI atoms by concatenation. No evaluation is performed; surfaces remain symbolic.

Classification and output. A workflow is PRUNED if its constraints contain a provable literal contradiction; CONDITIONAL if it requires runtime assignments

(guards, parameters, form validity, expression-bound gates); otherwise FEASIBLE. PRUNED workflows are excluded from the coverage denominator. A2 produces `a2-workflows.json` containing W; the output is deterministic and frozen.

3.4 B0: Subject Manifest

A2 produces symbolic constraint requirements—guard names, route parameter keys, form validity conditions—but cannot derive the concrete values needed to satisfy them: which account satisfies which guard, what entity identifier instantiates a route parameter, or how the application's login form is structured. These values are deployment-specific and external to the source code. B0 externalizes them as a per-subject manifest (`subject-manifest.json`), a versionable configuration artifact that supplies runtime bindings without modifying W or its constraint surfaces. This separation ensures that the workflow space remains source-derived while realization can adapt to different deployment environments.

The manifest contains: *accounts* mapping guard class names to credentials and roles; *route parameter values* binding symbolic keys to concrete values (with optional per-route-template overrides); *authentication setup* specifying login route, form selectors, and a post-login success indicator; *execution configuration* for B3 (readiness endpoints, timeouts, seed/reset commands); and optional *workflow exclusions*. Manifest authoring is assisted by a CLI wizard that reads A2 artifacts, collects referenced guards and parameter keys, and scaffolds a skeleton with placeholders. The completed manifest must pass schema validation before B1 consumes it. B0 is distinct from B1: B0 declares what values are available; B1 selects which values to bind to each workflow's constraints.

3.5 B1: Action Plan Derivation

B1 transforms each workflow into an executable *Action Plan* by binding symbolic constraints to manifest values, producing one plan per non-skipped workflow (`b1-plans.json`). It concretizes guards→accounts, parameters→values, and form fields→typed defaults; materializes preconditions (authentication, navigation, dialog entry); and selects element locators from A1 widget metadata using a fixed priority chain: `data-testid`, `id`, `formcontrolname`, `name`, `aria-label`, `routerlink`, `href`, `placeholder`, and positional fallback (`tag:nth-of-type`), each scoped to the owning component's selector when available. B1 plans every non-skipped workflow regardless of whether constraints are fully satisfiable, deferring failure to execution to preserve the denominator. Given identical W and manifest, plans are byte-identical.

3.6 B2: Code Generation

B2 generates one Selenium WebDriver[3] test file (TypeScript) per Action Plan, completing **RQ2**. Each test follows a fixed structure: precondition execution

[3] https://www.selenium.dev/documentation/webdriver/

(authentication, navigation, dialog activation), action steps (click, type, submit, select-option, wait), and postcondition assertion (URL-match or no-crash check). Each test embeds structured logging at generation time: upon completion, it writes a per-test execution log recording outcome, step-level evidence (identifiers, timestamps, URL state, element resolution, screenshots), and failure classification. This log is the primary evidence artifact for B3 integrity verification and failure taxonomy construction. B2 is deterministic; B3 consumes the generated tests unchanged.

3.7 B3: Evidence-Producing Execution

B3 executes each test against a live application, produces structured evidence, and verifies execution integrity, contributing to **RQ3**. Each test runs as an isolated subprocess with its own browser session; tests are sequential per subject. A readiness check precedes execution; unreachable applications yield FAIL_APP_NOT_READY for all workflows. Each test is attempted up to a configurable number of times (default: 1); the first passing attempt is accepted.

Outcome classification and integrity. B3 classifies outcomes into eight categories: PASS, FAIL_APP_NOT_READY, FAIL_ELEMENT_NOT_FOUND, FAIL_ASSERTION, FAIL_TIMEOUT, FAIL_AUTH, FAIL_INTEGRITY, and FAIL_UNKNOWN. For every apparent pass (exit code zero), B3 verifies integrity: the per-test execution log must exist, be fresh (postdating the run start), agree with the exit code, and contain at least one step entry. Any check failure downgrades the outcome to FAIL_INTEGRITY; a benchmark run with any such outcome invalidates C3. B3 produces `b3-results.json` with verified results, consumed by B4 and the diagnostic protocol.

3.8 B4: Coverage Computation

B4 computes tiered coverage over W, closing **RQ3**. Let $W^*_{exec} \subseteq W$ denote the executed workflows minus PRUNED, FAIL_APP_NOT_READY, and user-declared skips:

$$\text{C1} = |\{w \in W : \text{valid plan}\}| \,/\, |W| \tag{1}$$

$$\text{C2} = |\{w \in W : \text{valid test}\}| \,/\, |W| \tag{2}$$

$$\text{C3} = |\{w \in W^*_{exec} : \text{PASS}\}| \,/\, |W^*_{exec}| \tag{3}$$

C3 is reportable only with zero FAIL_INTEGRITY outcomes. B4 produces `b4-coverage.json`; given frozen inputs, the metrics are deterministic.

4 Evaluation

4.1 Setup

We evaluate the approach through three evaluation questions:

Table 1. Subject corpus. Columns report Angular version, authentication mechanism, A1 multigraph size (nodes/edges with structural-executable split), and A2 workflow count.

Subject	**Ng**	**Auth**	**Nodes**	**Edges**	**S/E**	$\|W\|$
posts-users-ui-ng	15	—	72	147	116/31	18
heroes-angular	11	—	67	88	68/20	19
softscanner-cqa	17	OAuth	42	84	67/17	16
airbus-inventory	12	JWT	68	149	108/41	21
spring-petclinic	18	—	195	430	302/128	74
ever-traduora	12	JWT	253	511	396/115	109
Total			697	1409	1057/352	257

- **EQ1** (RQ1): *Does Phase A produce a deterministic, auditable, bounded workflow denominator?*
- **EQ2** (RQ2): *Does Phase B achieve full generation coverage over that denominator?*
- **EQ3** (RQ3): *What execution coverage is achieved, how does it compare to a dynamic baseline, and what failure taxonomy emerges?*

The replication package (pipeline, artifacts, manifests, ground truth, protocols, evaluation data) is publicly available.[4]

Subject corpus. We evaluate on six open-source Angular applications, summarized in Table 1. Two subjects require JWT-based authentication; the remaining four are unauthenticated. One subject (softscanner-cqa) is included in the generation corpus (W) but excluded from the execution corpus (W_{exec}) because no benchmark-valid run was performed against a live instance.

Experimental protocols. The *determinism protocol* verifies byte-identical Phase A artifacts across independent runs (EQ1). The *benchmark execution protocol* defines valid B3 runs: canonical invocation, per-test evidence existence and freshness, exit-code/log agreement, zero FAIL_INTEGRITY, and documented metadata (EQ2–EQ3). The *diagnostic protocol* specifies multi-layer evidence inspection for failure classification (EQ3).

Ground truth. A ground truth (GT) set of 257 entries was constructed by a single auditor (∼40 hours) through systematic inspection of each subject's templates, event bindings, routing configurations, and handler bodies, cross-referenced against A1 multigraph edges and A2 workflow artifacts. For each trigger edge, the auditor verified that the corresponding A2 workflow correctly captured trigger kind, effect closure, terminal node, and constraint surface; ambiguities were resolved by tracing the full evidence chain (source → A1 → A2). GT construction is independent of execution outcomes: it validates denominator correctness, not test pass rates.

[4] https://github.com/softscanner-sw/softscanner_fa

4.2 EQ1: Workflow Space Construction

Determinism. Phase A extraction (A1) and enumeration (A2) were executed independently twice for each subject. In all cases, the output artifacts (`a1-multigraph.json`, `a2-workflows.json`) were byte-identical across runs, as verified by SHA-256 hash comparison.

Corpus statistics. Across six subjects, A1 extracted 697 nodes and 1,409 edges (1,057 structural, 352 executable). A2 enumerated 257 augmented workflows: 145 classified as FEASIBLE (56%) and 112 as CONDITIONAL (44%); zero workflows were PRUNED. The trigger distribution is dominated by handler triggers (170 WTH, 66%), followed by navigation (43 WNR, 17%), form submission (33 WSF, 13%), and external navigation (11 WNE, 4%).

GT validation. All 257 A2 workflows match the manually constructed ground truth (257/257). This confirms that the denominator is complete with respect to the audited trigger space and that no implemented trigger was missed or spuriously introduced.

Interpretation. EQ1 is answered affirmatively: Phase A produces a deterministic (byte-identical), auditable (source-traceable), and bounded (257 workflows) coverage denominator across all six subjects.

4.3 EQ2: Generation Coverage

B1 produced a valid Action Plan for all 257 workflows (C1 = 100%). B2 generated a syntactically valid Selenium test file for all 257 plans (C2 = 100%). The 257 generated tests contain 428 action steps (average 1.7 per test), with 106 tests (41%) requiring authentication preconditions and 9 (4%) requiring dialog-entry preconditions.

Performance. Phase A extraction and enumeration complete in under 10 seconds total across all six subjects (A1: ∼7s, A2: <1s), measured from pipeline JSONL logs. Plan derivation and code generation (B1+B2) complete in under 1 second for all 257 workflows. B3 execution time depends on subject complexity and retry configuration. Under the benchmark settings (maxRetries = 1), the full suite completes in ∼24 minutes (posts: 1.1 min, heroes: 1.0 min, airbus: 1.3 min, petclinic: 6.0 min, ever-traduora: 15.0 min). With maxRetries = 3, the two largest subjects increase substantially (petclinic: 15–25 min, ever-traduora: 55–64 min), as failed tests consume their full timeout budget on each retry. Generation cost (A1+A2+B1+B2: <10s) is negligible relative to execution cost, confirming that the computational bottleneck lies in runtime interaction.

Interpretation. EQ2 is answered affirmatively: the pipeline achieves full generation coverage over W in negligible time relative to execution. The generation bottleneck is fully removed; the remaining coverage gap (C3 < 100%) is attributable entirely to runtime execution barriers.

Table 2. Execution coverage (C3) per subject. softscanner-cqa is generation-only and excluded from W_{exec}.

Subject	Tests	Pass	Fail	C3
posts-users-ui-ng	18	16	2	88.9%
heroes-angular	19	17	2	89.5%
airbus-inventory	21	19	2	90.5%
spring-petclinic	74	55	19	74.3%
ever-traduora	109	49	60	45.0%
Total	**241**	**156**	**85**	**64.7%**

4.4 EQ3: Execution Coverage and Failure Taxonomy

Execution coverage. Table 2 reports per-subject results from the integrity-verified benchmark run (0 FAIL_INTEGRITY, 241/241 logs verified). Aggregate C3 is 64.7% (156/241); median per-subject C3 is 88.9%. The distribution is heterogeneous: three subjects exceed 88% (posts-users, heroes, airbus), spring-petclinic reaches 74.3%, while ever-traduora—the largest and most complex subject (109 workflows, JWT auth, 63 CONDITIONAL)—reaches 45.0% and contributes 60 of 85 total failures. The gap between aggregate and median reflects this concentration: ever-traduora alone accounts for 45% of the executed denominator and 71% of all failures.

Failure taxonomy. The 85 failures were classified through the diagnostic protocol into seven families (Table 3). The dominant family is *async-gate and repeater-readiness* (63/85, 74%): widgets behind asynchronous permission pipes or expression-gated visibility predicates that do not materialize within the implicit wait timeout. This family is concentrated in ever-traduora (52 failures) and spring-petclinic (9), both of which use runtime-resolved visibility extensively. The second family, *postcondition mismatch* (11/85, 13%), arises when the generated URL-match assertion does not account for non-navigating handler effects or server-side redirects. The remaining families—locator mismatch (6), external redirect (2), backend validation (1), dialog precondition (1), element not interactable (1)—account for 13% collectively.

Observability utility. The structured execution logs produced by B2-generated tests were essential for constructing the failure taxonomy. Of the 85 failures, 71 (84%) were classifiable from the per-test execution log alone (step-level failure kind, locator resolution, URL state before/after); 8 required additional inspection of screenshots or generated test code; and 6 required tracing back through the action plan, A2 workflow, and A1 multigraph to the application source. Without structured per-step evidence, failure diagnosis reduces to parsing raw stack traces, making systematic taxonomy construction impractical at this scale.

Structural comparison with AutoE2E. We compare structurally—not quantitatively—against AutoE2E [1], a crawling-based approach with LLM-driven feature inference. Direct C3 comparison is not possible: AutoE2E's implementation produced state graphs and feature databases but no executable test files for any subject. Meaningful exploration occurred only for spring-petclinic (9 states, 46 features);

Table 3. Failure taxonomy: 85 residual failures classified into seven families by diagnostic layer.

Family	Count	%	Layer
Async-gate / repeater-readiness	63	74	Runtime
Postcondition mismatch	11	13	Oracle
Locator mismatch	6	7	Structural
External redirect	2	2	Oracle
Backend validation	1	1	Environment
Dialog precondition	1	1	Structural
Element not interactable	1	1	Structural
Total	**85**	**100**	

authenticated subjects were blocked by authentication walls; remaining subjects hit a 30-action ceiling. Our approach produces 257 executable tests with a fixed denominator across all six subjects including authenticated ones. This asymmetry is inherent to the approaches' designs, not to our experimental setup.

4.5 Discussion

The results reveal a clear separation between generation and execution as coverage bottlenecks. Generation is fully solved: C1 = C2 = 100% across all subjects. Execution coverage varies substantially across subjects: median per-subject C3 is 88.9%, but aggregate C3 is 64.7% due to the disproportionate weight of evertraduora (109 workflows, 45% C3). For the four smaller subjects, execution coverage ranges from 74% to 91%, indicating that the pipeline is effective when asynchronous visibility gating is limited. The remaining gap is attributable to runtime barriers, not to planning or code generation.

The failure taxonomy identifies async-gate and repeater-readiness as the dominant execution barrier (74% of failures). These failures arise when widgets depend on asynchronously resolved data or permission state that does not materialize within the test's implicit wait window. This is a synchronization problem between generated test code and the application's runtime rendering pipeline, not a deficiency in workflow construction or constraint handling. Addressing it requires either adaptive wait strategies that account for async data resolution or deeper instrumentation of the application's change-detection lifecycle.

Because W is constructed from source code and fixed before execution, C3 measures genuine test-versus-application interaction, not exploration luck or model fidelity. The structural comparison with AutoE2E illustrates this: without a fixed denominator or executable test output, the baseline's coverage cannot be expressed over the same space.

4.6 Threats to Validity

Internal validity. Execution outcomes may be affected by transient runtime conditions (browser timing, network latency, application state). We mitigate this

through per-test browser isolation, bounded retry, integrity verification (zero FAIL_INTEGRITY), and pre-run stale-artifact cleanup. The benchmark execution protocol ensures that reported outcomes are evidence-backed.

Construct validity. Coverage is defined over a source-derived workflow space; this space may not capture all user-relevant behaviors (e.g., interactions involving dynamically loaded components not reachable via static analysis, or behaviors requiring multi-step stateful preconditions beyond the modeled constraint surface). The C3 postcondition oracle is URL-match, a weak assertion that verifies navigation but not DOM state or data correctness; C4 (oracle strength) is deferred to future work.

External validity. The evaluation covers six Angular applications; generalization to other frameworks or non-SPA architectures is not established. However, only the extraction front-end (A1) is Angular-specific, relying on Angular template syntax, decorators, and router configuration parsing. The downstream pipeline—workflow enumeration (A2), manifest binding (B0–B1), code generation (B2), execution (B3), and coverage computation (B4)—operates over framework-agnostic JSON artifacts and is not tied to Angular internals. Extension to similar TypeScript-based frameworks (React, Vue) would require a framework-specific A1 extractor but could reuse the remainder of the pipeline. Subject selection may not represent the full diversity of production Angular applications.

Conclusion validity. The AutoE2E comparison is structural, not quantitative (see §4.4). The aggregate C3 (64.7%) is sensitive to subject composition; excluding ever-traduora yields 81.3%. One subject (softscanner-cqa) contributes to C1/C2 but not C3.

5 Conclusion

Existing E2E testing approaches compute coverage over interaction spaces derived from external models or runtime exploration, yielding denominators that reflect abstraction fidelity or exploration budget rather than implemented behavior. This paper addresses that limitation with a source-driven, two-phase pipeline that statically extracts a UI Interaction Multigraph from Angular source code, enumerates a finite, constraint-aware workflow space W, and realizes every workflow into an executable Selenium test with structured observability and integrity-verified coverage computation.

Empirical evaluation on six Angular applications produced a workflow space of 257 augmented workflows with full generation coverage (C1 = C2 = 100%). Execution coverage on five integrity-verified subjects reached 64.7% (156/241), with a systematic failure taxonomy classifying 85 residual failures into seven families.

The central finding is that the generation bottleneck is fully resolved: every source-derived workflow receives a concrete, executable test. The dominant remaining barrier is runtime widget materialization—asynchronous permission gates and data-dependent visibility predicates that do not resolve within the

test's wait window—accounting for 74% of failures. This barrier is a synchronization problem between generated test timing and the application's rendering pipeline, not a deficiency in workflow construction or constraint handling.

Future work includes adaptive synchronization strategies for asynchronous UI gates, stronger postcondition oracles beyond URL-match (C4), extraction front-ends for additional component-based frameworks (React, Vue), and optional LLM-assisted data generation for CONDITIONAL workflows requiring complex runtime inputs. More broadly, the results demonstrate that E2E coverage can be measured against a fixed, implementation-grounded denominator derived directly from source code, providing a stable basis for comparing testing approaches independently of their exploration strategies.

References

1. Alian, P., Nashid, N., Shahbandeh, M., Shabani, T., Mesbah, A.: Feature-Driven End-to-End Test Generation. In: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. pp. 450–462. ICSE '25, IEEE Press, Ottawa, Ontario, Canada (Sep 2025). https://doi.org/10.1109/ICSE55347.2025.00141, https://doi.org/10.1109/ICSE55347.2025.00141
2. Balsam, S., Mishra, D.: Web Application Testing - Challenges and Opportunities. Journal of Systems and Software **219**, 112186 (Jan 2025). https://doi.org/10.1016/j.jss.2024.112186, https://www.sciencedirect.com/science/article/pii/S0164121224002309
3. Cavalcanti, A.R., Accioly, L., Valença, G., Nogueira, S.C., Morais, A.C., Oliveira, A., Gomes, S.: Automating Test Design Using LLM: Results from an Empirical Study on the Public Sector. Conference on Digital Government Research **26** (May 2025). https://doi.org/10.59490/dgo.2025.1025, https://proceedings.open.tudelft.nl/DGO2025/article/view/1025, conference Name: Conference on Digital Government Research
4. Chevrot, A., Vernotte, A., Falleri, J.R., Blanc, X., Legeard, B., Cretin, A.: Are Autonomous Web Agents Good Testers? Proceedings of the ACM on Software Engineering **2**(ISSTA), 206–228 (Jun 2025). https://doi.org/10.1145/3728879, https://dl.acm.org/doi/10.1145/3728879
5. Domínguez Osorio, J.M.: Automated GUI Ripping for Web Applications. Ph.D. thesis, Universidad de los Andes (2019), https://repositorio.uniandes.edu.co/handle/1992/44587, accepted: 2020-09-03T14:56:04Z Publisher: Uniandes
6. García, B., Dueñas, J.C.: Automated Functional Testing based on the Navigation of Web Applications. Electronic Proceedings in Theoretical Computer Science **61**, 49–65 (Aug 2011). https://doi.org/10.4204/EPTCS.61.4, http://arxiv.org/abs/1108.2357, arXiv:1108.2357 [cs]
7. Garousi, V., Keles, A.B., Balaman, Y., Guler, Z.O., Arcuri, A.: Model-based Testing in Practice: An Experience Report from the Web Applications Domain. Journal of Systems and Software **180**, 111032 (Oct 2021). https://doi.org/10.1016/j.jss.2021.111032, https://www.sciencedirect.com/science/article/pii/S0164121221001291
8. Garousi, V., Mesbah, A., Betin-Can, A., Mirshokraie, S.: A Systematic Mapping Study of Web Application Testing. Information and Software Technology **55**(8), 1374–1396 (Aug 2013). https://doi.org/10.1016/j.infsof.2013.02.006, https://www.sciencedirect.com/science/article/pii/S0950584913000396

9. Jensen, S.H., Madsen, M., Møller, A.: Modeling the HTML DOM and Browser API in Static Analysis of JavaScript Web Applications. In: Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering. pp. 59–69. ESEC/FSE '11, Association for Computing Machinery, New York, NY, USA (Sep 2011). https://doi.org/10.1145/2025113.2025125, https://dl.acm.org/doi/10.1145/2025113.2025125
10. Júnior, E., Valejo, A., Valverde-Rebaza, J., Neves, V.d.O.: GenIA-E2ETest: A Generative AI-Based Approach for End-to-End Test Automation. In: Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025). pp. 282–292 (Sep 2025). https://doi.org/10.5753/sbes.2025.9927, http://arxiv.org/abs/2510.01024, arXiv:2510.01024 [cs]
11. Leotta, M., Yousaf, H.Z., Ricca, F., Garcia, B.: AI-Generated Test Scripts for Web E2E Testing with ChatGPT and Copilot: A Preliminary Study. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering. pp. 339–344. ACM, Salerno Italy (Jun 2024). https://doi.org/10.1145/3661167.3661192, https://dl.acm.org/doi/10.1145/3661167.3661192
12. Li, T., Cui, C., Huang, R., Towey, D., Ma, L.: Large Language Models for Automated Web-Form-Test Generation: An Empirical Study — RCR Report. ACM Transactions on Software Engineering and Methodology p. 3797275 (Feb 2026). https://doi.org/10.1145/3797275, https://dl.acm.org/doi/10.1145/3797275
13. Li, T., Huang, R., Cui, C., Towey, D., Ma, L., Li, Y.F., Xia, W.: A Survey on Web Application Testing: A Decade of Evolution (Dec 2024). https://doi.org/10.48550/arXiv.2412.10476, http://arxiv.org/abs/2412.10476, arXiv:2412.10476 [cs]
14. Liu, C.H., You, S.D., Chiu, Y.C.: A Reinforcement Learning Approach to Guide Web Crawler to Explore Web Applications for Improving Code Coverage. Electronics **13**(2), 427 (Jan 2024). https://doi.org/10.3390/electronics13020427, https://www.mdpi.com/2079-9292/13/2/427, publisher: Multidisciplinary Digital Publishing Institute
15. Madsen, M., Livshits, B., Fanning, M.: Practical static analysis of JavaScript applications in the presence of frameworks and libraries. In: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering. pp. 499–509. ESEC/FSE 2013, Association for Computing Machinery (2013). https://doi.org/10.1145/2491411.2491417, https://doi.org/10.1145/2491411.2491417
16. Memon, A., Banerjee, I., Nagarajan, A.: GUI Ripping: Reverse Engineering of Graphical User Interfaces for Testing. In: Proceedings of the 10th Working Conference on Reverse Engineering. p. 260. WCRE '03, IEEE Computer Society, USA (Nov 2003)
17. Mesbah, A., Bozdag, E., van Deursen, A.: Crawling AJAX by Inferring User Interface State Changes. In: 2008 Eighth International Conference on Web Engineering. pp. 122–134 (Jul 2008). https://doi.org/10.1109/ICWE.2008.24, https://ieeexplore.ieee.org/abstract/document/4577876
18. Mesbah, A., van Deursen, A., Lenselink, S.: Crawling Ajax-Based Web Applications through Dynamic Analysis of User Interface State Changes. ACM Trans. Web **6**(1), 3:1–3:30 (Mar 2012). https://doi.org/10.1145/2109205.2109208, https://doi.org/10.1145/2109205.2109208
19. Nass, M., Alégroth, E., Feldt, R.: Why Many Challenges with GUI Test Automation (Will) Remain. Information and Software Technology **138**, 106625 (Oct 2021). https://doi.org/10.1016/j.infsof.2021.106625, https://www.sciencedirect.com/science/article/pii/S0950584921000963

20. Nebut, C., Fleurey, F., Le Traon, Y., Jezequel, J.M.: Automatic Test Generation: A Use Case Driven Approach. IEEE Transactions on Software Engineering **32**(3), 140–155 (Mar 2006). https://doi.org/10.1109/TSE.2006.22, https://ieeexplore.ieee.org/abstract/document/1610607
21. Oshima, N., Kishi, T.: Model Checking Method for SPA Page Transition Based on Component-based Framework. In: Proceedings of the 30th International Conference on Software Engineering and Knowledge Engineering (SEKE 2018). pp. 675–715 (Jul 2018). https://doi.org/10.18293/SEKE2018-038, http://ksiresearchorg.ipage.com/seke/seke18paper/seke18paper_38.pdf
22. Park, J., Sun, K., Ryu, S.: EventHandler-Based Analysis Framework for Web Apps Using Dynamically Collected States. In: Russo, A., Schürr, A. (eds.) Fundamental Approaches to Software Engineering. pp. 129–145. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-89363-1_8
23. Pastor Ricós, F., Slomp, A., Marín, B., Aho, P., Vos, T.E.J.: Distributed State Model Inference for Scriptless GUI Testing. Journal of Systems and Software **200**, 111645 (Jun 2023). https://doi.org/10.1016/j.jss.2023.111645, https://www.sciencedirect.com/science/article/pii/S0164121223000407
24. Shahbandeh, M., Alian, P., Nashid, N., Mesbah, A.: NaviQAte: Functionality-Guided Web Application Navigation (2024). https://doi.org/10.48550/ARXIV.2409.10741, https://arxiv.org/abs/2409.10741, version Number: 1
25. Sherin, S., Muqeet, A., Khan, M.U., Iqbal, M.Z.: QExplore: An Exploration Strategy for Dynamic Web Applications Using Guided Search. Journal of Systems and Software **195**, 111512 (Jan 2023). https://doi.org/10.1016/j.jss.2022.111512, https://www.sciencedirect.com/science/article/pii/S0164121222001881
26. Sung, C., Kusano, M., Sinha, N., Wang, C.: Static DOM Event Dependency Analysis for Testing Web Applications. In: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering. pp. 447–459. FSE 2016, Association for Computing Machinery, New York, NY, USA (Nov 2016). https://doi.org/10.1145/2950290.2950292, https://dl.acm.org/doi/10.1145/2950290.2950292
27. Utting, M., Legeard, B.: Practical Model-based Testing: A Tools Approach. Elsevier (2010)
28. Utting, M., Pretschner, A., Legeard, B.: A Taxonomy of Model-Based Testing Approaches. Software Testing, Verification and Reliability **22**(5), 297–312 (2012). https://doi.org/10.1002/stvr.456, https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.456, _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/stvr.456
29. Wang, C., Pastore, F., Goknil, A., Briand, L.C.: Automatic Generation of Acceptance Test Cases From Use Case Specifications: An NLP-Based Approach. IEEE Transactions on Software Engineering **48**(2), 585–616 (Feb 2022). https://doi.org/10.1109/TSE.2020.2998503, https://ieeexplore.ieee.org/abstract/document/9103626
30. Zhang, T., Hao, F., Wang, Y., Zhang, B., Xie, G.: VeriFlow: A Framework for the Static Verification of Web Application Access Control via Policy-Graph Consistency. Electronics **14**(18), 3742 (Jan 2025). https://doi.org/10.3390/electronics14183742, https://www.mdpi.com/2079-9292/14/18/3742
31. Zheng, Y., Liu, Y., Xie, X., Liu, Y., Ma, L., Hao, J., Liu, Y.: Automatic Web Testing Using Curiosity-Driven Reinforcement Learning. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). pp. 423–435 (May 2021). https://doi.org/10.1109/ICSE43902.2021.00048, https://ieeexplore.ieee.org/abstract/document/9402046, iSSN: 1558-1225